

IOWA STATE UNIVERSITY

Aircraft Data Visualization

Project Specification / HCI584 / Andy Walker

Project Introduction

In the past, radar research has been limited to large companies and those with exceptional budgets. However, Amateur Radio enthusiasts are increasingly able to buy and build capable microwave systems for communications, sharing of data, and sensing of obstacles. These capabilities are primed to push amateur electronics into the realm of real microwave sensors like radars. Radars use radio signals to detect and track 'targets' in their field of view, and larger targets make for easier, more certain detections. Commercial aircraft provide excellent targets of opportunity for such a system. They are physically and electrically large, they move at high speed and broadcast their position regularly via radio links that can be used to provide a source of truth for the hobbyist measurement. This enables educational radar training by providing a test bed for unclassified, amateur radio compatible radar.

Details of Design

This project will use API pulls to collect ADS-B (https://en.wikipedia.org/wiki/Automatic_Dependent_Surveillance%E2%80%93Broadcast) data from aircraft that are currently less than a specified distance from a central location. Given the GPS coordinates of that central location and the coordinates of the aircraft as calculated from the ADS-B data, the tool will calculate an azimuthal angle and elevation angle that constitute a pointing vector (i.e. line of sight) from the central location to the aircraft. Any aircraft too small to be detected by a presumed sensor e.g. a low-cost radar sensor, will be excluded from the list of sensible aircraft.

Phase 1 will focus on the basics of the API data pull, data formatting, and core user interaction. The calculations will compute the line of sight from the sensor to the aircraft and output the azimuth and elevation angles from the central location to the target aircraft. Topological data will be used to calculate any obstruction of the pointing vector. Phase 2 will add a user interface to the product of Phase 1. This GUI will enable visualization of real-time calculations of lines of sight to aircraft inside a sensible volume. The interface will also allow the user to modify the parameters of the hypothetical sensor to increase or decrease the number of the aircraft discovered.

The graphical user interface plots the aircraft computed to be accessible by the presumed sensor on a map centered on the coordinates of the central location. The map will include range rings showing a 'stoplight gauge' (green-yellow-red) of ranges

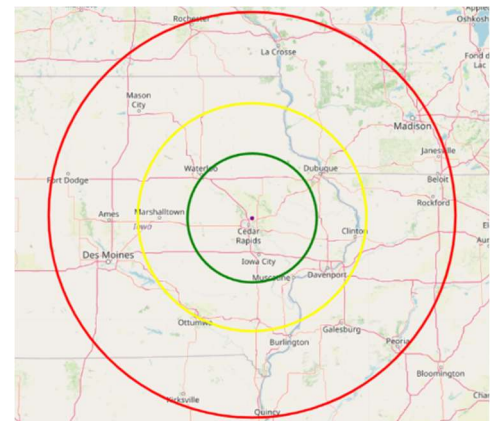


Figure 1: Map with Stoplight Gauge

(Figure 1) that correspond to the minimum size aircraft expected to be detected by the sensor. This GUI should evolve over time, showing the aircraft flying along their routes, and as they come into or fall out of the accessible area, the aircraft should appear or disappear accordingly. While an aircraft location is plotted, clicking on or hovering over the indicator should show identification of the aircraft, e.g. tail number, flight number, etc.

Use Cases (Vignettes)

The vignettes are broken into 2 categories of 2 types each. The first division is whether the presumed radar system is operating in a 'monostatic' or 'multi-static' mode. In this case, monostatic' means that there is only one sensor node that both illuminates the target with radio waves and also detects the echoes of those waves. Conversely, multi-static' means that one location illuminates the targets while nodes at other locations listen for the echoes. This matters to the algorithms as well as the determination of locations. The second case captures whether the sensor is detecting a target or tracking a known target.

• Major Task: Monostatic Detection

In this case, the user determines their location using GPS or selects a predefined location to use as the location of the single-node sensor. This is the location of the illuminator as well as the echo sensor. To simulate the sensor, the ADS-D data and GPS data are used to calculate a line of sight from the central

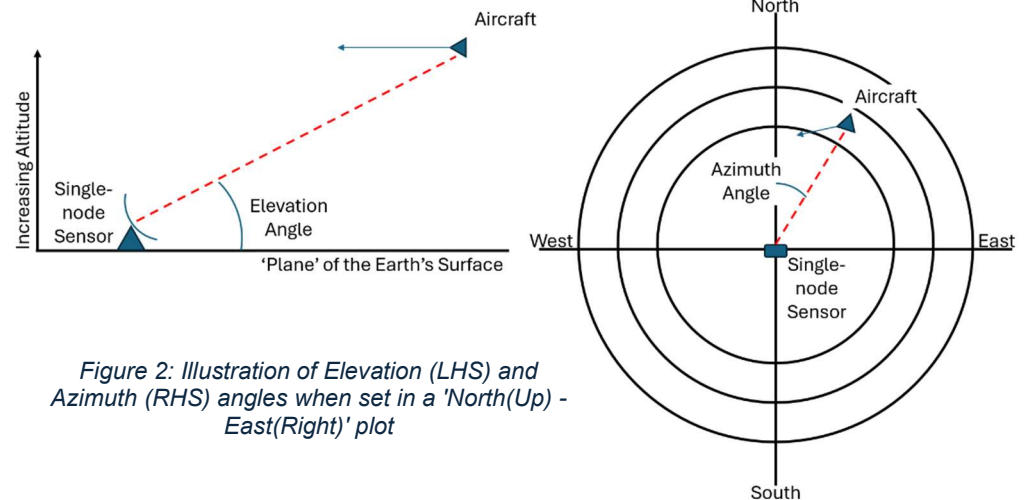


Figure 2: Illustration of Elevation (LHS) and Azimuth (RHS) angles when set in a 'North(Up) - East(Right)' plot

location to the aircraft. Along that line, any obstructions of the line of sight are calculated leveraging an API pull or database of elevations. Finally, given no obstructions, the azimuth and elevation angles are returned to the user for subsequent calculations.

Order of Tasks:

- 1) Determine Central Location
- 2) Collect ADS-B data
- 3) Calculate line of sight vector
- 4) Calculate obstructions along line of sight
- 5) If none, return azimuth and elevation angles to aircraft

• Major Task: Monostatic Target Tracking

In actual sensor operation in the absence of ADS-B data, the beam (Figure 2, red dotted line) would be swept in a regular pattern attempting to location echoes of large aircraft. This is extremely time-consuming and energy intensive, and therefore once a target is discovered, it is advantageous for the sensor to track the location of every detected aircraft possibly in tracking range. Because aircraft fly on common trajectories, a simple linear extrapolation of the trajectory will be provided to estimate the future location of the aircraft (Figure 2, blue solid line).

Order of Tasks:

- 1) Perform Detection (see above)
- 2) Start Loop
- 3) Estimate linear trajectory
- 4) Collect ADS-B data
- 5) Compare estimate to current position/altitude
- 6) Update trajectory
- 7) End Loop

• Major Task – Multi-Static Detection

In multi-static scenarios, the user determines the location of multiple sensor nodes using high-precision (RTK) GPS or selects a predefined location to use as the location of the illuminator and one or more sensors. Note that the illuminator can be collocated with one sensor, but the multi-static use case requires at least one more node to also sense echoes.

To simulate the sensor, the ADS-B data and GPS data are used to calculate a line of bearing from each sensor location and the illuminator location to the aircraft. Along those lines, any obstructions of the line of sight are calculated leveraging an API pull or database of elevations. Finally, given no obstructions, the azimuth and elevation angles are returned to the user for subsequent calculations.

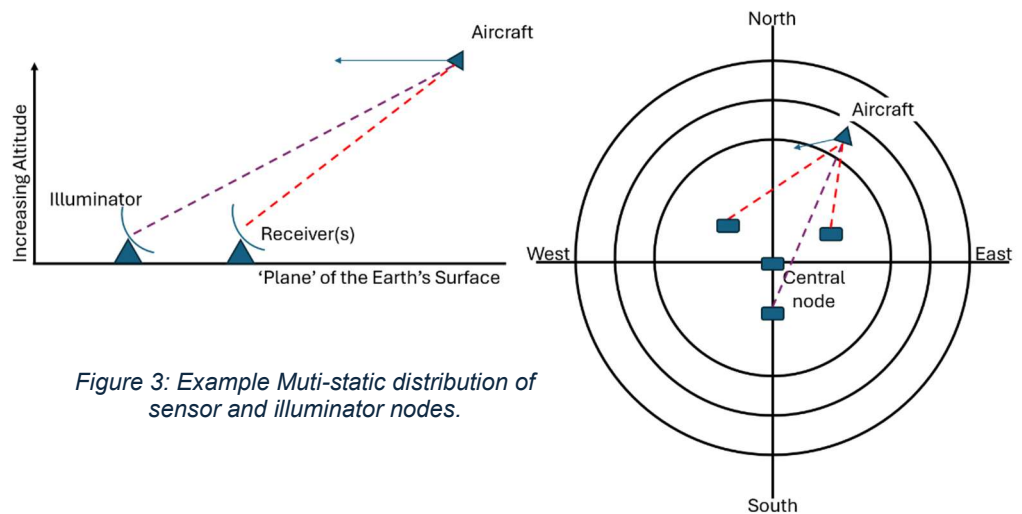


Figure 3: Example Multi-static distribution of sensor and illuminator nodes.

Order of Tasks:

- 1) Determine Location of 2 or more nodes
- 2) Collect ADS-B data
- 3) Calculate line of sight vectors
- 4) Calculate obstructions along lines of sight
- 5) If none, return azimuth and elevation angles to aircraft

• Major Task – Multi-Static Target Tracking

As before, in operation and in the absence of ADS-B data, the beam (Figure 2, red dotted line) would be swept in a regular pattern attempting to location echoes of large aircraft. This is extremely time-consuming and energy intensive, and therefore once a target is discovered, it is advantageous to the sensor to track the location of every aircraft *relative to the illuminator and receiver(s)*. Because aircraft fly on common trajectories, a simple linear extrapolation of the trajectory will be provided to estimate the future location of the aircraft (Figure 2, blue solid line).

Order of Tasks:

- 1) Perform Detection (see above)
- 2) Start Loop

- 3) Estimate linear trajectory
- 4) Collect ADS-B data
- 5) Compare estimate to current position/altitude
- 6) Update trajectory

Technical Flow

A simplified data flow is shown in Figure 4. Once inside the main loop, the ADS-B data is collected each iteration, as is the GPS data. Finally, map data is retrieved to facilitate plotting of the ranges from the central node to the aircraft. Note that mobility of the sensor(s) is supported by this model. For fixed site systems, no updates are required for GPS locations and therefore map data. The ADS-B API responds with a dictionary of lists of dictionaries. The map API responds with an image. The serial device responds with a text stream from a hardware device that can be captured and formatted by common python libraries. Once a few basic inputs from the user are entered (e.g. selecting real-time or recorded data), the main loop runs until broken.

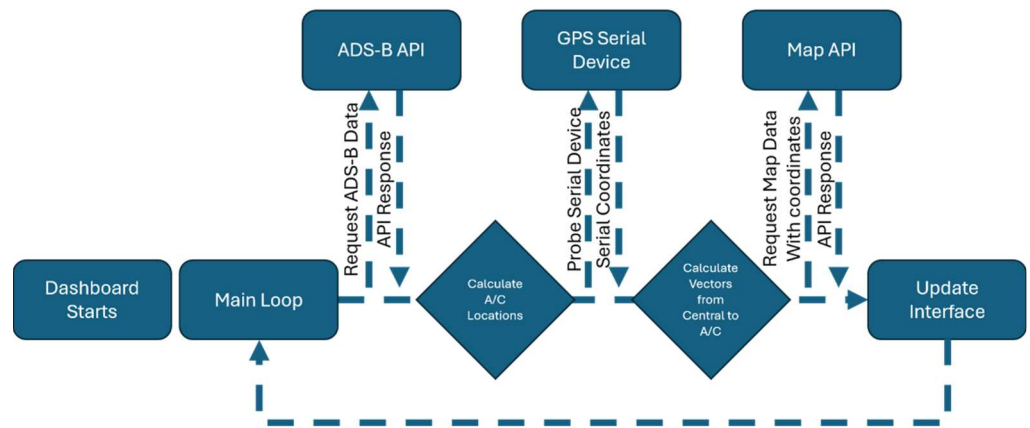


Figure 4: Simplified Data Flow

Figure 5 shows a nominal graphical user interface with the map located top center. The rest of the messages and controls are for the physical sensor and not used (yet).

To support the calculations and the graphical user interface development, this project expects to use commonly available libraries. Listing the candidates in no particular order:

http.client and **urllib** (To help collect API responses)
pyproj (for Transformer, which converts coordinates into different formats)
requests (to make API requests)

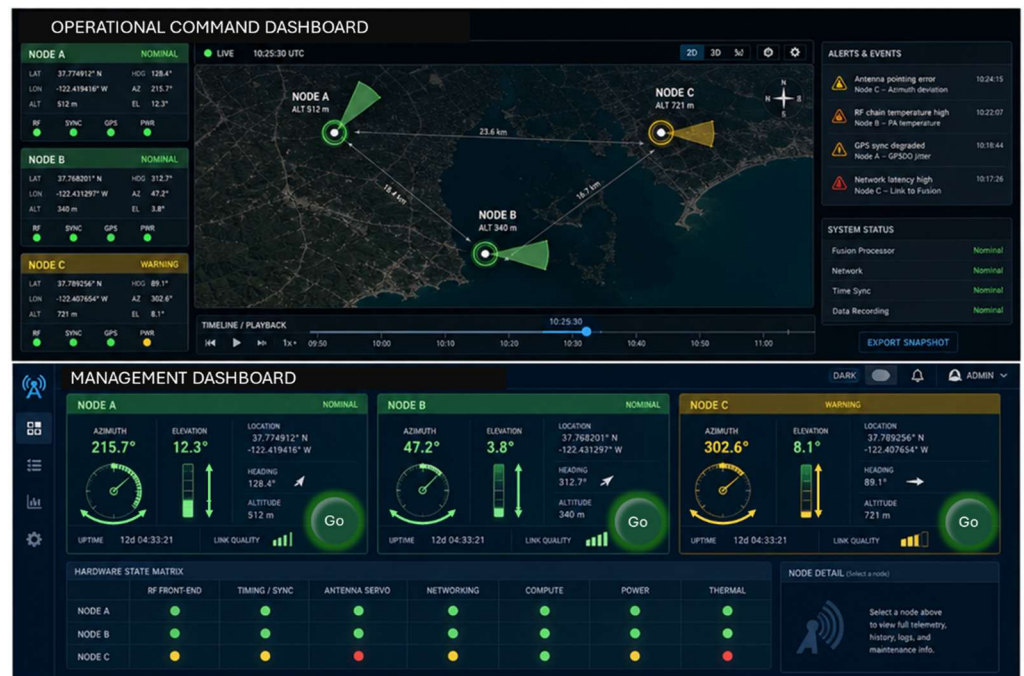


Figure 5: Model User Interface (Dark Theme)

serial (for collecting the GPS data from the serial device) and **pynmea2** (to format the GPS data) **folium** (for free map data) and **openmeteo_requests** (for free elevation data) **streamlit**, **streamlit_folium**, and **folium** to handle GUI work and **pandas**, **datetime**, **numpy**, and **math** to round out the basic range of libraries. In Phase 2, update rates will be of critical importance and preliminary work with Streamlit have generated GUIs with very jerky updates.

Requirements

Phase 1

Requirement 1: The application **shall** pull real-time data from an ADS-B API service.

Requirement 2: The application **shall** compute the line-of-sight vector from the sensor node(s) to the aircraft 'target' using the sensor node(s) locations in 3-space and the target location in 3-space.

Objective 1: The application **will** pull the location of the sensor node(s) via GPS serial streams.

Requirement 3: The application **shall** calculate the slant range to the aircraft 'target' from the sensor location.

Requirement 4: The application **shall** calculate the radar range from the central node for 4 different size targets (1, 10, 100, 1000 square-meter targets)

Requirement 5: The application **shall** limit the range of addressed aircraft with a variable set by a number not to exceed 100 miles from the central node.

Requirement 6: The application **shall** compute terrain masking along the line-of-sight vector from the central node to each aircraft.

Objective 2: The application **will** pull elevation data from a free API to ensure no limitations on data availability.

Requirement 7: The application **shall** compute a trajectory for each aircraft to be used as a tracking hypothesis.

Objective 3: The application **will** use a simple linear extrapolation for each aircraft based on historic flight data over time.

Phase 2

Requirement 1: The application **shall** pull map data from an API centered on the central node latitude and longitude.

Objective 1: The application **will** pull free map data to ensure unlimited API pulls

Requirement 2: The application **shall** plot the map data in a web-based application graphical user interface.

Requirement 3: The application **shall** plot range circles for the 4 different size targets (1, 10, 100, 1000 square-meter) on the web-based GUI in Requirement 2.

Requirement 4: The application **shall** plot the line-of-sight vectors from the central node to each aircraft on the web-based GUI from Requirement 2.

Requirement 1: The application **shall** update faster than 1 time per second.

Objective 2: The application **will** draw the aircraft data without redrawing the maps, range rings, etc. to minimize flickering.

Follow-On work

Enable Real-Time Kinematic (RTK) data corrections for ultra-accurate central location measurement

The Iowa Real Time Network (<https://iowadot.gov/consultants-contractors/design/iowa-real-time-network>) is Iowa's global navigation satellite system (GNSS) reference station network. The goal of the network is to provide a series of fixed-site (surveyed) GNSS (e.g. GPS) receivers that publish the measurements that those receivers make. Since the receivers know where they are with high precision and don't move, the errors they measure can be broadcast on the internet and used to correct for atmospheric disturbances to GPS receivers that are mobile. Specialized receivers called Real-Time Kinematic (RTK) receivers apply the corrections broadcast by IARTN to improve GPS receiver accuracy to within 2 inches.

Putting 2 of these receivers on a short boom (roughly 36") attached to the radar antenna enables 2 separate high precision measurements that can be used to calculate the azimuth and elevation pointing angles of the antenna. We can then know the precise location of the antenna on the surface of the earth as well as which direction it's pointed and if there is any tilt in the boom relative to the surface of the earth. The state of Iowa provides open geospatial APIs and datasets to develop custom software, building scripts, or mapping applications:

Conclusion

This specification captures the requirements and objectives of the first phase of the algorithm and graphical user interface (GUI) development. Remaining work is captured in the Follow-On section.

CONTACT INFORMATION

Any questions can be directed to the author, Andy Walker, at the following:

- andyw@iastate.edu
- andywalker2001@hotmail.com
- 319-538-4032

After working through the spec, what was the biggest or most unexpected change you had to make from your sketch? – I hadn't considered the tracking use case and while a linear extrapolation is useful for commercial airliners, smaller hobby planes will have more erratic flight paths and a more involved tracking algorithm may be required.

How confident do you feel that you can implement the spec as it's written right now? – I think I have the bones of this now, and keeping a log of the locations over time to make a state vector for each aircraft seems like the logical next step.

What is the biggest potential problem that you NEED to solve (or you'll fail)? – I think the biggest problem I've seen so far is the flickering in the GUI. I'll need help to make a smoothly updating web app.

What parts are you least familiar with and might need my help? – The GUI and controls pieces, although Python libraries sure seem to help...